CS 61A Structure and Interpretation of Computer Programs Midterm 3

Fall 2026

Solutions

Tuesday, October 27, 2026

Print Your Name: _____

Print Your Student ID: _____

Print Your @berkeley.edu Email: _____

Print Your Exam Room: _____

Print the Name of Person to your Left: _____

Print the Name of Person to your Right: _____

Instructions

You have **1 hour 50 minutes** to complete the exam. There are **8 questions** and **10 pages** on this exam, including this cover page.

Question	1	2	3	4	5	6	7	8	Total
Points	5	5	7	9	7	9	3	0	45

- The exam is closed book, closed notes, closed computer, closed calculator, except two 8.5" × 11" pages of your own creation and the provided Midterm 1, Midterm 2, and Midterm 3 Study Guides.
- Mark your answers on the exam itself in the spaces provided. We will not grade answers outside the designated answer spaces.
- To use the restroom, bring your exam and phone to the front of the room.
- You may use built-in Python functions that do not require import, such as `pow`, `len`, `abs`, `bool`, `int`, `float`, `str`, `round`, `max`, `min`, `list`, `tuple`, `sum`, `all`, `any`, `map`, `filter`, `zip`, `sorted`, `reversed`, `iter`, and `next`.
- You **may not** use example functions defined on your study guide unless a problem clearly states you can.
- Unless otherwise specified, you are allowed to reference functions defined in previous parts of the same question.
- You may not use `;` to place two statements on the same line.
- You may use the `Link` class defined on the Midterm 2 Study Guide.
- You may use the `Tree` class and `is_leaf` function defined on the Midterm 3 Study Guide.

Questions with **circular bubbles**: you may select only **1 choice**.

○ Unselected option (completely unfilled)

● Single option selected (completely filled)

Questions with **square boxes**: you may select **1 or more choices**.

■ You may select multiple squares

■ as long as they are completely filled

You must fill in the bubbles **completely**. Ticks, crosses, or other check marks will **not** receive credit.

Honor Code

"As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others."

Sign Your Name: _____

This page intentionally left blank

The exam begins on the next page.

1. [5.0 points] What Would Python Display?

Assume the following code has been executed.

```
def odds(t):
    for x in t:
        if x % 2 == 1:
            yield x

def echo(prefix):
    def f(x):
        print(prefix, x)
        return x
    return f

nums = range(1, 5)
g = map(echo('outer'), odds(map(echo('inner'), nums)))
```

For each expression below, identify what the interactive Python interpreter would display after evaluating it. These expressions are evaluated in order, so **evaluating each expression could affect the result of evaluating later expressions**.

(a) [1.0 pt] What is displayed by `print(sum(odds(nums)))`?

- ○ `0`
- ○ `1`
- ○ `3`
- ● `4`
- ○ `6`
- ○ `9`
- ○ `10`
- ○ `15`

(b) [2.0 pts] Select all **lines** that are displayed by `print(next(g))`. If no lines are displayed, select *Nothing*.

- ■ `inner 1`
- ☐ `inner 2`
- ☐ `inner 3`
- ☐ `inner 4`
- ☐ `inner 5`
- ■ `outer 1`
- ☐ `outer 2`
- ☐ `outer 3`
- ☐ `outer 4`
- ☐ `outer 5`
- ■ `1`
- ☐ `2`
- ☐ `3`
- ☐ `4`
- ☐ `5`
- ☐ *Nothing*
- ☐ ERROR (including `StopIteration`)

(c) [2.0 pts] Select all **lines** that are displayed by evaluating `print(next(g))` again. If no lines are displayed, select *Nothing*.

- ☐ `inner 1`
- ■ `inner 2`
- ■ `inner 3`
- ☐ `inner 4`
- ☐ `inner 5`
- ☐ `outer 1`
- ☐ `outer 2`
- ■ `outer 3`
- ☐ `outer 4`
- ☐ `outer 5`
- ☐ `1`
- ☐ `2`
- ■ `3`
- ☐ `4`
- ☐ `5`
- ☐ *Nothing*
- ☐ ERROR (including `StopIteration`)

2. [5.0 points] Under the Limit

In the large answer box on the right, write what is displayed by this code. It's recommended, but not required, that you draw an environment diagram in the space below. The answer is the same whether it is run in a file or in an interactive interpreter.

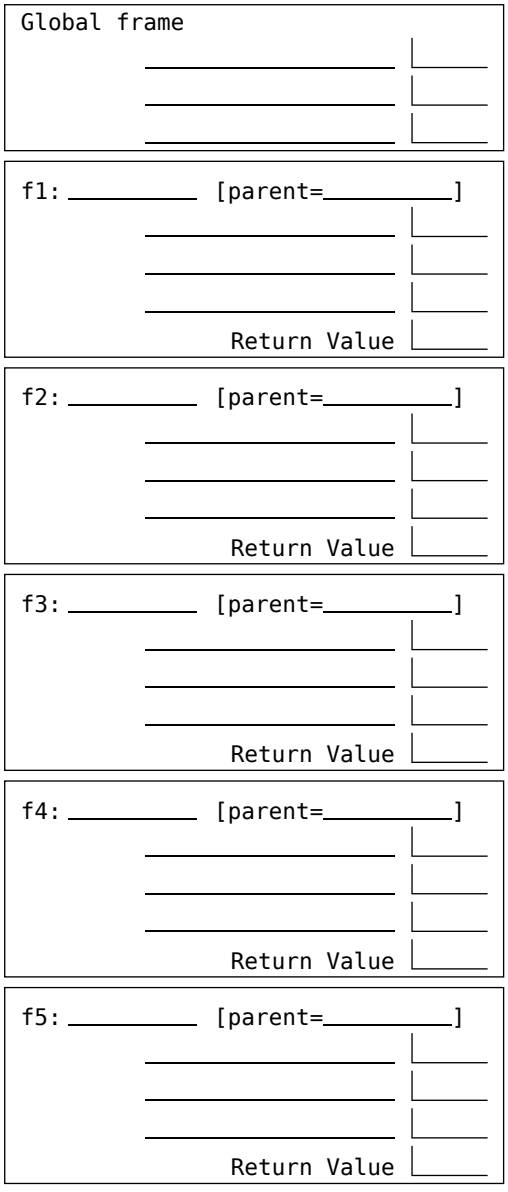
This code does not run forever or error. You may not need all of the frames in the diagram.

```
default = []
def under(s, limit):
    if s == None:
        s = default
    def f(t):
        while t.pop(0) < limit:
            s.append(t.pop(0))
    return f

s = [3, 1, 4, 2, 5, 9]
under(None, 5)(s[2:])
under(None, 4)(s)
print([default, s])
```

Write your answer here:

```
[[2, 1], [2, 5, 9]]
```



3. [7.0 points] Spooky Memoize

Implement `memoize`, which takes a function `f` and returns a memoized version of `f`. Assume `f` is a one-argument function that takes a non-negative integer and returns an integer. The memoized version of `f` should call `f` at most once for each unique argument and only on the arguments passed to it. The return value of `f(n)` is stored in the list `crypt` at index `n`, and `crypt` contains `None` at each index `n` less than its length for which `f(n)` has not been called.

```
def memoize(f):
    """Return a memoized version of f.
    >>> def square(x):
    ...     print('square', x)
    ...     return x * x
    >>> sq = memoize(square)
    >>> sq(3)
    square 3
    9
    >>> sq(3)
    9
    >>> [sq(5), sq(1), sq(3)]
    square 5
    square 1
    [25, 1, 9]
    """
    crypt = []  # like a cache, but spookier!
    def memo_f(n):
        if _______:
             (a)
            crypt._______(_______)
                    (b)     (c)
        if crypt[n] == None:
            _______
              (d)
        return _______
                  (e)
    return memo_f
```

[1.0 pt] Choose the correct choice to fill in blank (a).

- ◯ `n <= 0`
- ◯ `f(n) <= n`
- ◯ `crypt[0] <= n`
- ● `len(crypt) <= n`
- ◯ `n not in crypt`
- ◯ `crypt[n] == None`
- ◯ `crypt[-1] <= n`
- ◯ `n + len(crypt) <= 0`

[1.0 pt] Choose the correct choice to fill in blank (b).

- ◯ `pop`
- ◯ `append`
- ● `extend`
- ◯ `insert`
- ◯ `remove`

[2.0 pts] Fill in blank (c).

`[None for i in range(n - len(crypt) + 1)]` *(or `[None] * (n - len(crypt) + 1)`)*

[2.0 pts] Choose the correct choice to fill in blank (d).

- ◯ `crypt[n] = n`
- ◯ `crypt[-1] = n`
- ◯ `crypt.append(f(n))`
- ◯ `crypt.insert(n, f(n))`
- ● `crypt[n] = f(n)`
- ◯ `crypt[-1] = f(n)`
- ◯ `crypt.append([f(n)])`
- ◯ `crypt.insert(n, [f(n)])`
- ◯ `crypt[n] = memo_f(n)`
- ◯ `crypt[-1] = memo_f(n)`
- ◯ `crypt = f(n)`
- ◯ `crypt = crypt + f(n)`
- ◯ `crypt[n] = f(crypt[n])`
- ◯ `crypt[-1] = f(crypt[-1])`

[1.0 pt] Choose the correct choice to fill in blank (e).

- ◯ `n`
- ◯ `f(n)`
- ◯ `crypt`
- ● `crypt[n]`

4. [9.0 points] Flip Sum

Definition: The *flip sum* of a list adds its even-indexed items and subtracts its odd-indexed items. For example, the flip sum of `[4, 1, 6, 2, 7]` is `4 - 1 + 6 - 2 + 7 = 14`. The flip sum of `[]` is 0.

(Q4.1) Implement `max_flip`, which takes a list of **positive** numbers `s` and returns the largest flip sum of any list that can be formed by removing zero or more items from `s` (without changing the order of the remaining items).

```
def max_flip(s: list[int]) -> int:
    """Return the largest flip sum within a list of positive numbers s.

    >>> max_flip([3, 1, 4])                    # 3 - 1 + 4
    6
    >>> max_flip([2, 9, 5])                    # 9
    9
    >>> max_flip([4, 1, 1, 6, 2, 7])           # 4 - 1 + 6 - 2 + 7
    14
    >>> max_flip([3, 9, 2, 6, 7, 4, 3, 1, 8])  # 9 - 2 + 7 - 1 + 8
    21
    """
    if not s:
        return 0
    return max(s + _______ + [_______ for j in range(1, len(s))])
                   (a)         (b)
```

[2.0 pts] Choose the correct choice to fill in blank (a).

- ◯ `max_flip(s[1:])`
- ◯ `s[0] + max_flip(s[1:])`
- ◯ `s[0] - max_flip(s[1:])`
- ● `[max_flip(s[1:])]`
- ◯ `[s[0] + max_flip(s[1:])]`
- ◯ `[s[0] - max_flip(s[1:])]`

[3.0 pts] Fill in blank (b).

`s[0] - s[j] + max_flip(s[j+1:])` *(ok to write `s[j:]`)*

[1.0 pt] What is the purpose of including `s` in the expression `max(s + ...)` in the recursive case?

- ◯ To include the flip sum of `s` with nothing removed.
- ◯ To include every list that starts with `s[0]`.
- ● To include every list of length one formed from `s`.
- ◯ To include every list that does not contain `s[0]`.

(Q4.2) [3.0 pts] Implement `flip_link`, which takes a linked list of numbers `s` **with an even length** and returns its flip sum. The `Link` class appears on the Midterm 2 Study Guide. **Your answer must be one line of code.**

```
def flip_link(s: LinkedList[int]) -> int:
    """Return the flip sum of a linked list of numbers s.

    >>> flip_link(Link(4, Link(1, Link(6, Link(2)))))   # 4 - 1 + 6 - 2
    7
    >>> flip_link(Link(5, Link(3)))                     # 5 - 3
    2
    """
    if not isinstance(s, Link):
        return 0
    assert isinstance(s.rest, Link)
```

`return s.first - s.rest.first + flip_link(s.rest.rest)`

5. [7.0 points] TreeDoh

The `Tree` class and `is_leaf` function appear on the Midterm 3 Study Guide.

(Q5.1) Definition: A root-to-leaf path is *increasing* if every label along it is larger than the label before it.

[3.0 pts] Implement `inc`, which returns the number of increasing paths in a `Tree` `t`. Your answer must be **one** line of code.

```
def inc(t: Tree[int]) -> int:
    """Return the number of increasing paths from the root of t to a leaf.
    >>> t = Tree(3, [Tree(5, [Tree(4), Tree(6), Tree(8)]), Tree(2, [Tree(7)]), Tree(4)])
    >>> inc(t)   # 3-5-6, 3-5-8, and 3-4
    3
    >>> inc(Tree(5, [Tree(1), Tree(2)]))
    0
    """
    if is_leaf(t):
        return 1
```

```
return sum([inc(b) for b in t.branches if b.label > t.label])
```

(Q5.2) Definition: The *last leaf* of a tree `t` is the leaf whose label is printed last when `print(t)` is called. For example, the node labeled 0 is the last leaf in `Tree(1, [Tree(2, [Tree(3), Tree(4)]), Tree(0)])`.

Implement `no_last`, which takes a `Tree` `t` **with at least two nodes** and returns a tree identical to `t` but without its last leaf. Do not mutate `t`.

```
def no_last(t: Tree) -> Tree:
    """Return a tree identical to t but without its last leaf.
    >>> t = Tree(1, [Tree(2, [Tree(3), Tree(4)]), Tree(5, [Tree(6, [Tree(7)])])])
    >>> print(no_last(t))
    1
      2
        3
        4
      5
        6
    >>> print(no_last(no_last(no_last(t))))   # removes 7, then 6, then 5
    1
      2
        3
        4
    """
    if is_leaf(_______):
                (a)
        return Tree(t.label, t.branches[:-1])
    return Tree(t.label, _______)
                           (b)
```

[1.0 pt] Choose the correct choice to fill in blank (a).

- ◯ `t`
- ◯ `t.branches`
- ◯ `t.branches[0]`
- ● `t.branches[-1]`
- ◯ `no_last(t.branches[0])`
- ◯ `no_last(t.branches[-1])`

[3.0 pts] Fill in blank (b).

```
t.branches[:-1] + [no_last(t.branches[-1])]
```

6. [9.0 points] Tracker

A `Tracker`, like an Apple AirTag, tracks locations but does not have `GPS`. Instead, it can request a location from a nearby `Phone`. It has an `owner` (`str`) and a `location`. Its `update_location` method requests the location of a nearby `Phone` and stores it.

A `GPS` has a `locate` attribute that is a zero-argument function that returns a location (a two-element tuple).[1]

A `Phone` stores its `GPS` as a value in a dictionary called `gps`, keyed by `'p@ssw0rd'`. It also has a list of the `trackers` that have requested its location (one `Tracker` per request). The `request_location` method prints a warning if the tracker has requested this phone's location before, records the tracker, and returns a location from its `GPS`. The warning begins with the `Phone`'s `warn` class attribute, which is different for `LoudPhone`.

```
class Tracker:
    """Requests locations from nearby phones and stores the most recent location it has received.

    >>> t = Tracker('Oski')
    >>> p = Phone(GPS([(37.94, -122.35), (37.77, -122.42), (37.44, -122.14)]))
    >>> q = LoudPhone(GPS([(37.34, -121.89), (37.87, -122.27), (37.55, -121.99)]))
    >>> t.update_location(p)
    >>> t.update_location(q)
    >>> t.update_location(p)
    warning: a tracker owned by Oski requested your location again!
    >>> t.update_location(q)
    ALERT: a tracker owned by Oski requested your location again!
    >>> Tracker('Oski').update_location(p)  # Oski bought a new tracker, so no warning
    >>> t.location
    (37.87, -122.27)
    """
    def __init__(self, owner: str):
        self.owner = owner
        self.location = None

    def update_location(self, phone: Phone) -> None:
        self.location = _______
                          (a)

class GPS:
    def __init__(self, locations):
        self.locate = locations.pop

class Phone:
    warn = "warning"
    def __init__(self, gps):
        self.gps = {'p@ssw0rd': gps}
        self.trackers = []

    def request_location(self, t: Tracker) -> tuple[float, float]:
        s = _______
              (b)
        if t in s:
            print(f'{_______}: a tracker owned by {_______} requested your location again!')
                        (c)                            (d)
        s.append(t)
        return _______
                 (e)

class LoudPhone(Phone):
    warn = "ALERT"
```

[1] An actual GPS (Global Positioning System) receiver computes its location from timing signals broadcast by satellites. This one just pops from a list.

[2.0 pts] Fill in blank (a). **Your answer must call `request_location`.**

```
phone.request_location(self)
```

[1.0 pt] Choose the correct choice to fill in blank (b).

- ◯ `[]`
- ◯ `trackers`
- ◯ `t.trackers`
- ● `self.trackers`
- ◯ `[t]`
- ◯ `Phone.trackers`
- ◯ `tracker.trackers`
- ◯ `Tracker.trackers`

[1.0 pt] Select all choices that correctly fill in blank (c).

- ☐ `warn`
- ■ `self.warn`
- ☐ `Phone.warn`
- ☐ `LoudPhone.warn`
- ☐ `t.warn`
- ☐ `super.warn`
- ☐ `Tracker.warn`
- ☐ `super().warn`

[1.0 pt] Select all choices that correctly fill in blank (d).

- ☐ `t`
- ☐ `owner`
- ☐ `self.owner`
- ☐ `Phone.owner`
- ☐ `s`
- ■ `t.owner`
- ☐ `self.t.owner`
- ☐ `Tracker.owner`

[2.0 pts] Choose the correct choice to fill in blank (e).

- ◯ `gps['p@ssw0rd'].locate`
- ◯ `gps['p@ssw0rd'].locate()`
- ◯ `self.gps['p@ssw0rd'].locate`
- ● `self.gps['p@ssw0rd'].locate()`
- ◯ `Phone.gps['p@ssw0rd'].locate`
- ◯ `Phone.gps['p@ssw0rd'].locate()`
- ◯ `self['p@ssw0rd'].locate`
- ◯ `self['p@ssw0rd'].locate()`
- ◯ `gps['p@ssw0rd'].locate(self)`
- ◯ `self['p@ssw0rd'].locate(self)`
- ◯ `self.gps['p@ssw0rd'].locate(self)`
- ◯ `Phone.gps['p@ssw0rd'].locate(self)`

[2.0 pts] The `hack` function returns a location from a phone's `GPS` without using the secret `p@ssw0rd` and without calling `request_location`. Fill in blank (f). You **may not** write quotation marks or `p@ssw0rd` or `request_location`.

```
def hack(phone: Phone):
    """Return the location of a phone without calling request_location.

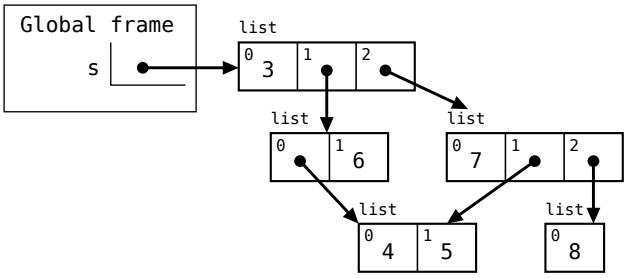
    >>> p = Phone(GPS([(37.91, -122.07), (37.86, -122.49)]))
    >>> hack(p)
    (37.86, -122.49)
    >>> hack(p)
    (37.91, -122.07)
    >>> p.trackers
    []
    """
    return next(_______).locate()
                (f)
```

```
iter(phone.gps.values())
(or map(phone.gps.get, phone.gps) or (phone.gps[k] for k in phone.gps))
```

7. [3.0 points] Box and Pointer

The box-and-pointer diagram shows the list `s` after both lines of this code are executed.

```
s = [3, [[4, 5], 6]]
s.append(_______)
           (a)
```



[2.0 pts] Fill in blank (a) so that executing the code produces this environment diagram.

```
[7, s[1][0], [8]]
```

[1.0 pt] What is displayed by `print(s[2])`?

```
[7, [4, 5], [8]]
```

8. Count Boxes

This A+ question is worth no points; it can only raise a high A to an A+. Finish the rest of the exam first!

Definition: A nested list of numbers is a list that contains only numbers and nested lists of numbers.

Fill in `count_boxes`, which takes a nested list of numbers `s` and returns the number of boxes in its box-and-pointer diagram.

Note: `[4] in [[4]]` evaluates to `True`. **Restrictions:** You may **not** use `else`, `set`, `dict`, `frozenset`, `eval`, `exec`, `id`, `:=`, `{` or `}`.

```
def count_boxes(s: list) -> int:
    """Return the number of boxes in the box-and-pointer diagram for s.

    >>> count_boxes([3, 4, 5])
    3
    >>> t = [4, 5]
    >>> count_boxes([t, t])            # t is drawn once
    4
    >>> count_boxes([[4, 5], [4, 5]])  # two different lists
    6
    >>> u = [t, t, t]
    >>> v = [u, u[0], u + [4]]
    >>> count_boxes(v)
    12
    """
    seen = []  # each list counted so far
    def count(s: list) -> int:
        seen.append(s)
        return len(s) + _______
    return count(s)
```

Fill in the blank. If your answer is long, write it on multiple lines.

```
sum([count(y) for y in s if type(y) == list and all([y is not z for z in seen])])
```